

MIPS Floating-Point Unit and Matrix Multiplication

1 Floating-Point Co-Processor (FPU) Overview

- MIPS FP hardware is implemented as **coprocessor 1**.
- It is an adjunct processor that extends the MIPS ISA.
- Includes a separate set of FP registers.

2 Registers

- **32 single-precision registers:** \$f0, \$f1, ..., \$f31
- Registers can be paired for double-precision:
 - Example: \$f0/\$f1, \$f2/\$f3, ...
 - Only even-numbered registers are used as base addresses.
- FP instructions only operate on FP registers.
- Mixing integer and FP data is discouraged.

2.1 Register Convention

\$f0 - \$f3	Return values
\$f4 - \$f11, \$f16 - \$f19	Temporaries
\$f12 - \$f15	Arguments
\$f20 - \$f31	Saved registers

3 FP Instructions in MIPS

3.1 Arithmetic

- Single-precision: `add.s`, `sub.s`, `mul.s`, `div.s`
- Double-precision: `add.d`, `sub.d`, `mul.d`, `div.d`
- Example:

```
1 mul.d $f4, $f4, $f6
```

Listing 1: Floating Point Multiplication

3.2 Load/Store

- Single: `lwc1`, `swc1`
- Double: `ldc1`, `sdc1`
- Base address comes from integer registers.
- Example:

```
1 ldc1 $f8, 32($sp)
```

Listing 2: Double-Precision Load

3.3 Comparison and Branching

- FP compare: `c.xx.s`, `c.xx.d` where `xx` = `eq`, `lt`, `le`, ...

```
1 c.lt.s $f3, $f4
```

Listing 3: FP Compare

- Branch: `bc1t`, `bc1f`

```
1 bc1t Label
```

Listing 4: FP Branch

4 Note

- No FP immediate instructions.
- Use global memory to store constants and access via `$gp`.

5 Example: Fahrenheit to Celsius

C Code:

```
1 float f2c(float fahr) {  
2     return ((5.0f / 9.0f) * (fahr - 32.0f));  
3 }
```

MIPS Code:

```
1 f2c:  
2     lwc1 $f16, 36($gp) # Load 5.0  
3     lwc1 $f17, 40($gp) # Load 9.0  
4     div.s $f16, $f16, $f17 # f16 = 5.0 / 9.0  
5     lwc1 $f17, 44($gp) # Load 32.0  
6     sub.s $f17, $f12, $f17 # f17 = fahr - 32  
7     mul.s $f0, $f16, $f17 # f0 = result  
8     jr $ra
```

6 Example: Matrix Multiplication

C Code:

```
1 for (i = 0; i != 32; i++)  
2     for (j = 0; j != 32; j++)  
3         for (k = 0; k != 32; k++)  
4             x[i][j] += y[i][k] * z[k][j];
```

6.1 Registers Used

- i: \$s0, j: \$s1, k: \$s2
- x: \$s3, y: \$s4, z: \$s5

6.2 Address Calculation

- Each matrix has 32×32 elements of 8 bytes each.
- Index: $i \times 32 + j$
- Address: $\text{Base} + (i * 32 + j) * 8$

6.3 MIPS Loop Skeleton

```

1 addi $s0, $zero, 0 # i = 0
2 L0:
3     addi $t0, $zero, 32
4     beq $s0, $t0, LOEND
5
6     addi $s1, $zero, 0 # j = 0
7 L1:
8     addi $t0, $zero, 32
9     beq $s1, $t0, L1END
10
11    addi $s2, $zero, 0 # k = 0
12 L2:
13    addi $t0, $zero, 32
14    beq $s2, $t0, L2END
15
16    # Computation block here
17
18    addi $s2, $s2, 1
19    j L2
20 L2END:
21    addi $s1, $s1, 1
22    j L1
23 L1END:
24    addi $s0, $s0, 1
25    j L0
26 LOEND:

```

6.4 MIPS Computation Block

```

1 # x[i][j] = x[i][j] + y[i][k] * z[k][j];
2 # x[i][j]
3 sll $t0, $s0, 5
4 add $t0, $t0, $s1
5 sll $t0, $t0, 3
6 add $t0, $s3, $t0
7 ldc1 $f16, 0($t0)
8
9 # y[i][k]
10 sll $t0, $s0, 5
11 add $t0, $t0, $s2
12 sll $t0, $t0, 3
13 add $t0, $s4, $t0
14 ldc1 $f18, 0($t0)
15
16 # z[k][j]
17 sll $t0, $s2, 5

```

```
18 add $t0, $t0, $s1
19 sll $t0, $t0, 3
20 add $t0, $s5, $t0
21 ldc1 $f20, 0($t0)
22
23 # Multiply and accumulate
24 mul.d $f18, $f18, $f20
25 add.d $f16, $f16, $f18
26
27 # Store back x[i][j]
28 sll $t0, $s0, 5
29 add $t0, $t0, $s1
30 sll $t0, $t0, 3
31 add $t0, $s3, $t0
32 sdc1 $f16, 0($t0)
```